



DRISHTI

PRODUCT REQUIREMENTS DOCUMENT (PRD)

Drishti — Product Requirements Document (PRD)

Core Vision, User Personas, Feature Specifications & Success Metrics



Graph-Powered Attack Paths



Live Watch Agent



Dollar Risk Quantifier



AI Remediation Guardrails



URL Trust Scoring

PLATFORM VERSION

v0.2 Enterprise Edition

RELEASE DATE

August 2026

ARCHITECTURE TIER

Full-Stack Multi-Tenant



RESTRICTED // INTERNAL ARCHITECTURE & TECHNICAL SPECIFICATION // FOR AUTHORIZED USE ONLY

Drishti — Product Requirements

This document records the WHAT and WHY of the implemented product — goals, capabilities, user journeys, non-functional requirements, constraints, and success metrics — reverse-engineered from the implemented code. It is a ground-truth inventory, not design intent.

Last updated: 2026-08-17 — verified against source code.

1. Product vision



NOTE

"Don't patch what you can't reach."

Drishti is an AI-powered network risk simulator for small SOC teams, pentesters, and engineers. It answers the question: **"Which host in my network is the cheapest to own?"** — in dollar terms, in ranked attack paths, and on an attack map.

2. Core value proposition

Pillar	User sees	Drishti provides
Attack map	Topology of real hosts + exposed services + CVE findings on a React Flow graph	Server-side layered layout, deterministic, live-updating
Attack paths	Ranked paths from INTERNET to each exposed asset, in hop order	Yen k-shortest, risk + likelihood + \$ impact per path
Dollar exposure	A single headline number: total exposed value	Computed from the engine (not the LLM) — deterministic
AI remediation	Per-finding fixes (Ansible / shell / cloud-CLI / manual)	LLM-generated, output-guarded, human-in-the-loop
Live network view	Force map of real LAN devices, gateway, threats	Agent-discovered devices + ARP-spoof/rogue/threat detection
Hardening report	Per-node quantified reduction projections	Engine-measured: PATCH / VLAN / ISOLATE projections

3. Capability inventory (by screen)

3.1 Dashboard

- Total exposure (\$) — engine sum of max path-impact per unique target
- Open findings count
- Critical asset count
- Top path risk (0–100)
- Top paths table (entry, target, hops, risk, likelihood, \$ impact, narrative)
- Zone breakdown (inventory + exposure + avg risk)
- Severity distribution (critical / high / medium / low)

3.2 Attack Map

- React Flow graph: INTERNET → DMZ → Internal → Cloud → Crown Jewel
- Nodes: assets (color-coded by risk score), gateway, live devices, threat nodes (pulsing red)
- Edges: color-coded by `relation` (`network` , `admin` , `trust` , `exposure`)
- Path highlighting: `onTopPath` flag + `path_id` on edges
- Sidebar: node detail (services, findings, blast radius)
- Focus mode: click a node to see its blast radius highlighted
- Crown jewel marker (star icon) on critical assets
- Offline device overlay: online devices hang off gateway; stale → offline dot

3.3 Live Watch

- Force-directed graph: gateway → devices (WiFi/ARP-discovered)
- Node state: online/offline (90s stale window), MAC, vendor, subnet
- Threats: ARP-spoof, rogue device, risky service, malicious domain — with MITRE tags
- Demo attack button: injects labeled synthetic threats for demo/onboarding
- Domain check: on-demand URL Trust Analyzer for observed domains
- Block/fix: produce a block rule or remediation for a risky domain

3.4 Paths

- Ranked path list (entry → target, hop count, risk, likelihood, \$ impact)
- Path detail: hop-by-hop walkthrough with CVE + edge weight per hop
- Blast radius: downstream reachable assets + combined value

3.5 Findings

- Open / remediating / resolved / accepted status filters
- Severity filter
- Patch action: transitions status → triggers recompute
- Remediate action: opens AI remediation flow

3.6 Remediation Console

- Per-finding AI-generated fix (Ansible playbook / shell / cloud-CLI / manual)
- Script copy-to-clipboard
- Review flag (human-in-the-loop)
- Regenerate (force new LLM call)
- Disclaimer banner (AI-generated, validate in your environment)
- Full-screen error view with retry/skip when the LLM is unavailable

3.7 Report

- CVE table: all CVEs affecting the org
- Severity distribution chart
- ML summary: model stats (call counts, mock vs real)
- Hardening report: per-node, measured % reduction for PATCH / VLAN / ISOLATE actions

3.8 URL Analyzer

- URL input + analysis
- Score gauge (0–100)
- Severity band: Trusted / Caution / High Risk
- Per-provider breakdown (SSL, WHOIS, domain age, content, Safe Browsing, VirusTotal)
- History: past analyses with URL, score, band

3.9 Settings

- Agent mode toggle (ingest / observe / devices)
- Agent log viewer
- Network map toggle (show/hide)
- Agent token display (masked)
- Deep-scan config

3.10 Demo

- Demo attack controls
- Inject/clear demo attack

3.11 Admin

- Load sample network (Acme Retail, \$902,900 baseline)
- Reset org data
- Rotate agent token
- Member list

4. Data sources (four input channels)

4.1 Agent Ingest (POST /api/ingest)

- **Source:** Edge agent (`drishti_watch.py` or `drishti_agent.py`)
- **Scope:** Host + services + vulnerabilities + connectivity
- **Auth:** Agent token (hashed, SHA256)
- **Idempotency:** `(org_id, ip)` for assets, `(asset_id, port, protocol)` for services, `(asset_id, vulnerability_id)` for findings, `(from, to, relation)` for connections
- **Race-safe:** SAVEPOINT + adopt-concurrent-row pattern
- **Side effect:** Triggers `recompute_org()`

4.2 Deep Scan (POST /api/live/deep-scan)

- **Source:** User-initiated (frontend → API) or Autoscan (background)
- **Scope:** Single IP or CIDR ($\leq /22$)
- **Auth:** User JWT
- **Consent:** Explicit `consent: true`; RFC1918-only
- **Capabilities:** nmap -sV (version detection) + NVD CVE lookup
- **Honest absence:** `{available: false, unavailable_reason: "..."} if scan/lookup fails`

4.3 Netconfig Audit (POST /api/netconfig/analyze)

- **Source:** User-initiated (frontend → API)
- **Scope:** DMZ / NAT / DHCP detectors against current topology
- **Auth:** User JWT
- **Output:** `{status: "real" | "unknown" | "passed", source: "observed" | "declared", evidence}`

4.4 Live Telemetry (agent devices mode)

- **Source:** Edge agent in devices mode
- **Scope:** ARP/ping sweep → IP, MAC, hostname, vendor, subnet
- **Auth:** Agent token
- **On-device observability:** DNS queries, active tabs/apps, network connections
- **Side effect:** `POST /api/live/devices` → `NetworkDevice` rows → threat detection

4.5 URL Trust

- **Source:** User (browser or extension) or agent domain observation
- **Scope:** Single URL or domain
- **Auth:** User JWT
- **Providers:** SSL, WHOIS, domain age, content heuristics (always); Google Safe Browsing, VirusTotal (optional, keyed)

5. User journeys

5.1 New user → first insight

1. Sign up (/signup) → Organization + Admin user created
2. Sees landing → landing page has 5 CTA cards (each opens a panel)
3. Either: a. Click "Simulate your own" → sees fresh empty state (map/dashboard show no data) b. Admin loads sample network → sees Acme Retail, \$902,900

5.2 Agent → auto-enriched topology

1. Admin gets agent token (Settings)
2. Deploys edge agent with token
3. Agent runs in ingest mode → sends host + services + vulns + connectivity
4. Backend upserts → triggers recompute
5. Dashboard updates: \$ exposure, top paths, findings, zone breakdown

5.3 Finding → remediation

1. SOC analyst sees a critical finding on the Attack Map
2. Click finding → opens detail
3. "Generate fix" → AI produces remediation (Ansible playbook / shell)
4. Review the script → mark as reviewed → apply in environment
5. Recompute → exposure drops → dollar figure updates

5.4 Live watch → threat detection

1. Agent runs in devices mode → discovers LAN hosts
2. Backend detects ARP spoofing (two devices claim gateway MAC)
3. Threat shows on the map: red pulse on the gateway node
4. Mitigation guidance: set static ARP entry

5.5 Deep scan → CVE discovery

1. User clicks a device → "Deep Scan"
2. Consents → nmap runs → CVEs looked up
3. Findings appear → attack paths update → dollar exposure increases
4. Analyst triages: remediate or accept risk

6. Non-functional requirements

6.1 Correctness

- Engine is the **single source of truth** for all derived values (risk scores, paths, dollars)
- Engine state is **recomputed** (not incrementally updated) on every trigger

- Engine values are **cached** in DB rows, never estimated from the UI

6.2 Safety

- No offensive code path in codebase — every LLM call is wrapped in GUARDRAIL
- No exploit check / agentic run / payload generation in the scanner
- Demo attack data is labeled and clearable
- Agent token is returned once; hash-only storage

6.3 Resilience

- Failure surfaces honestly (`available: false`, `unknown`, `configured: false`)
- AI failure surfaces as `{refused: true, reason: " ... "}` with a retry/skip UI
- Deep scan failure surfaces as `{unavailable_reason: " ... "}` with mitigation guidance
- LLM timeout: 45 s; fails gracefully to canned fixtures

6.4 Performance

- Recompute is a single pass: build graph → score nodes → enumerate paths → compute impact → write cache
- Path enumeration is bounded (Yen's k-shortest with hop / candidate / top-K caps)
- User-facing `/paths` is read from cache (cheap)
- Code-split frontend: landing/auth never download React Flow

6.5 Privacy

- URL Trust Analyzer: provider responses are **never cached** (response-only)
- URL Trust Analyzer result IS cached (History) — but full `UrlAnalysisResult` (signals + website + providers) is stored
- Live observations are domain-only (not full URL paths)
- Device inventory is IP/MAC only (no traffic content)
- The agent discloses its mode and scope

6.6 Portability

- SQLite default for local dev; Postgres in production (psycopg v3)
- UUIDs as 36-char strings (Postgres + SQLite compatible)
- No Alembic; `reconcile_columns` handles additive schema changes

7. Constraints

7.1 Scope constraints

- **No exploit execution:** the scanner discovers + looks up CVEs; it never tests or runs exploits
- **No outbound scanning without consent:** deep-scan is RFC1918-only + explicit consent

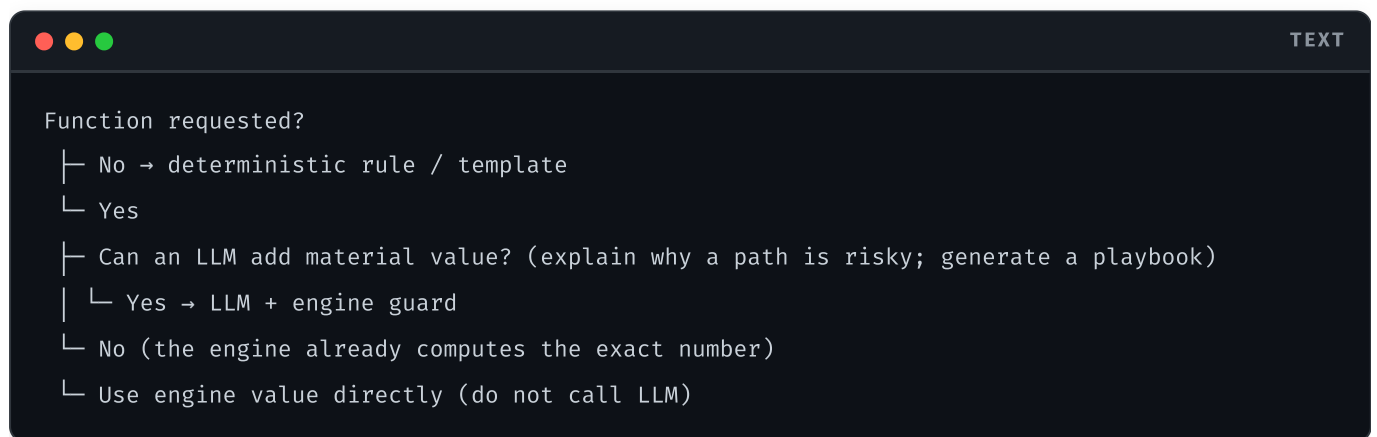
- **No data aggregation across orgs**: engine processes one org at a time
- **No persistent PII beyond email**: users table has no phone / address / external identity
- **Demo mode is optional**: fresh boot seeds identity only; demo network requires `DEMO_SEED=1`

7.2 Technical constraints

- Max deep-scan CIDR: `/22`
- Max concurrent deep-scan hosts: 32 per batch, 256 total
- Body size cap: 1 MB
- AI request timeout: 45 s
- Max AI tokens: 2500
- Agent token: base64url-encoded, 24 bytes
- Path enumeration: bounded by hop count (10), candidates (30), top-K (25)

8. AI integration patterns

8.1 Decision tree for AI use



8.2 Engine-authoritative dollars

- `POST /api/ai/impact`: engine pre-computes `impact_usd`; AI narrates but **endpoint overwrites** the model's number
- `total_exposure` in the dashboard: engine sum of max path-impact per unique target (read from cached `attack_paths`)

8.3 AI output pipeline

1. Cache check: return last remediation unless `regenerate=true`
2. Build context: asset, service, vulnerability details
3. Deterministic template as fallback (context-specific)
4. LLM call (or mock) with structured JSON schema
5. Output guard: scan for offensive markers → return `{refused: true, reason}`

6. Empty-script guard: never persist an empty fix
7. Persist: `Remediation` row, `reviewed=False`, disclaimer stamp

9. Compliance posture

Regulation	Application	Status
SOC 2	Access control, audit logs, change management	Partial — JWT auth + org isolation + structured logs
GDPR	Data minimization, right to deletion	Organization → reset (<code>POST</code> <code>/api/org/reset</code>). No PII beyond email
GDPR §5(d)	Purpose limitation	No cross-org data sharing
CWE	Remediation content references CWE IDs	Remediation scripts include CWE mapping
NIST CSF	Identify, Protect, Detect	Identify (assets, findings, paths), Protect (remediation), Detect (live threats)

10. Success metrics

Metric	How measured	Target
Time to first insight	New org → sample network loaded → attack map renders	< 30s
Recompute latency	<code>/stats</code> endpoint <code>recompute_ms</code>	< 500ms
AI availability	<code>/stats</code> endpoint <code>ai_calls</code> vs <code>ai_mock_calls</code>	> 95% real (mock < 5%)
AI refusal rate	<code>{refused: true}</code> responses / total AI calls	< 2%
Finding resolve → exposure drop	Dollar diff between pre/post recompute	Matches engine value (deterministic)
Deep-scan CVE discovery rate	CVE rows added per scan / NVD lookup success	> 80%
Live device accuracy	<code>NetworkDevice.is_self</code> / <code>is_gateway</code> / vendor match	Matches actual hardware
Threat detection latency	ARP-spoof / rogue detected → surface in UI	< 90s stale window
Code coverage	Vitest coverage report	Target 70%+

11. Open product questions (not addressed in code)

Question	Context
How does onboarding work for first-time users (no agent, no sample)?	Fresh boot is identity-only; the UI shows empty dashboards. No guided tour exists.
Can users share threat intelligence between orgs?	<code>threat_intel</code> table exists but is unused in v1 flows. The <code>ttp_tags</code> and <code>source</code> columns hint at a future Web3-style federated sharing.
What is the intended pricing model?	No billing or subscription code exists. The product appears to be community/evaluation-focused.
How do teams collaborate on findings?	No commenting, assignment, or ticket-integration workflow exists. Each user sees the same org data.

12. Feature parity matrix

Feature	Implemented	Notes
Org + user management	Yes	Admin/analyst/viewer roles
Agent ingest (idempotent)	Yes	Upsert + reconcile + recompute
Risk engine (NX)	Yes	Scores, blast radius, Yen paths, \$ impact
Attack map (React Flow)	Yes	Deterministic layout, live-gate, threats
Attack paths + \$	Yes	Ranked, hop-by-hop, cached
AI remediation	Yes	GUARDRAIL, human-in-the-loop, 3 providers
AI impact narrate	Yes	Engine-authoritative dollars
AI predict	Yes	Forward-looking narrative
AI network summary	Yes	Executive narrative
Live network (LAN devices)	Yes	WiFi-aware, 90s stale
Domain observation + URL Trust	Yes	Multi-provider, never caches API responses
Threat detection	Yes	ARP-spoof, rogue, risky service, malicious domain
Deep scan + CVE lookup	Yes	Consent-gated, RFC1918, honest absence
Netconfig audit	Yes	DMZ/NAT/DHCP detectors
Autoscan	Yes	Per-org schedule, subscan consent
Hardening report	Yes	Measured per-node projections
Telegram alerts	Yes	Background daemon, new findings + threats
Sample network (Acme)	Yes	\$902,900 baseline, triggered by <code>DEMO_SEED=1</code>
Demo attack mode	Yes	Labeled, injectable, clearable
Blast radius	Yes	Per-asset downstream count + value
URL history	Yes	History page with replay